

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes

Sección 20

Jorge Yass



Proyecto 1

Uso de un protocolo existente

Ana Laura Tschen 221645

Guatemala, 8 de septiembre

Descripción del proyecto

El proyecto consiste en el desarrollo de un servidor Model Context Protocol (MCP) que se ejecuta de forma local y permita a un chatbot empresarial consultar la información almacenada en una base de datos SQL Anywhere. Asimismo, se puede ejecutar de manera remota en una instancia de Google Cloud para consultar la información de una base de datos anonimizada derivada de la base de datos de SQL Anywhere en PostgreSQL. El caso de uso está orientado a directivos y responsables de la operación, quienes podrán formular preguntas en lenguaje natural relacionadas con ventas, productos, movimientos, líneas de producto y otros indicadores disponibles.

El chatbot actuará como anfitrión y utilizará un modelo de lenguaje para comprender las solicitudes del usuario y seleccionar las herramientas apropiadas. El servidor MCP será responsable de validar los parámetros, ejecutar consultas controladas sobre la base de datos y devolver resultados estructurados. El modelo de lenguaje no tendrá acceso directo a la base ni podrá ejecutar consultas SQL arbitrarias.

Objetivo General

El objetivo general del proyecto es implementar un chatbot que funcione como MCP Host y que pueda interactuar con distintos servidores MCP locales y remotos utilizando JSON-RPC 2.0. Como objetivos específicos se planteó implementar el ciclo de vida básico de MCP, conectar el chatbot con un LLM mediante API, mantener el contexto de una conversación, desarrollar un servidor MCP propio para análisis empresarial, integrar los servidores Filesystem y Git, implementar una versión remota del servidor, registrar las interacciones MCP y analizar los protocolos utilizados en la comunicación de red.

Ejemplo de uso

Pregunta del usuario:

“¿Qué productos tuvieron una reducción importante en ventas durante el último mes?”

Flujo esperado:

El chatbot envía la pregunta y el contexto al modelo de lenguaje.

El modelo selecciona una herramienta de análisis y proporciona parámetros estructurados.

El cliente MCP construye una solicitud JSON-RPC y la envía al servidor local.

El servidor valida los parámetros y ejecuta una consulta previamente controlada.

La base SQL Anywhere devuelve los registros requeridos.

El servidor calcula o resume los indicadores y devuelve una respuesta estructurada.

El modelo transforma los resultados en una explicación comprensible para el usuario

Stack de tecnologías utilizado

- Python: desarrollo del chatbot, cliente MCP, servidor MCP y lógica empresarial.
- LiteLLM: integración unificada con la API del modelo de lenguaje.
- SAP SQL Anywhere 17: sistema gestor de la base de datos empresarial.
- Drivers de SQL Anywhere para Python: conexión segura entre el servidor MCP y la base de datos.
- SQL: elaboración de consultas parametrizadas y controladas.
- JSON-RPC 2.0: formato de comunicación implementado manualmente para MCP.
- Model Context Protocol, MCP: protocolo utilizado para exponer e invocar herramientas.
- Vue.js: desarrollo de la interfaz web del chatbot.
- Vite: entorno de desarrollo y construcción del frontend.
- TypeScript: lógica de la interfaz.
- HTML y CSS: estructura y diseño visual.
- HTTP/HTTPS: comunicación entre el frontend, el backend y el servidor MCP remoto.
- Git y GitHub: control de versiones, documentación y entrega del proyecto.
- Google Cloud: para levantar la instancia remota del MCP y la BD anonimizada
- Docker: facilita los despliegues en la instancia remota
- Wireshark: captura y análisis de la comunicación entre el cliente y el servidor MCP remoto

Arquitectura General

La arquitectura está compuesta por una interfaz desarrollada en Vue, una API FastAPI, un administrador de sesiones, un orquestador de conversaciones, LiteLLM como capa de integración con modelos de lenguaje, un cliente MCP, servidores MCP locales y remotos, herramientas empresariales, bases de datos y un sistema de auditoría. El modo local utiliza comunicación mediante stdin/stdout, mientras que el servidor remoto utiliza HTTP/HTTPS. La arquitectura mantiene separadas las responsabilidades de presentación, razonamiento, comunicación MCP, ejecución de herramientas y acceso a datos, facilitando la reutilización de los componentes y reduciendo el acoplamiento entre ellos.



Figura 1. Diagrama de Arquitectura

Implementación del chatbot

La integración con el modelo de lenguaje se encuentra encapsulada mediante LiteLLM, lo que permite seleccionar distintos proveedores utilizando variables de entorno. Cada conversación se administra mediante una sesión independiente que conserva el historial de mensajes, permitiendo mantener contexto entre preguntas consecutivas. El orquestador envía al LLM la conversación y los schemas de las herramientas disponibles; cuando el modelo determina que necesita una operación externa, genera una llamada de herramienta que posteriormente es transformada en una solicitud MCP. Además, el sistema registra las interacciones entre cliente y servidor incluyendo dirección, método, ID, transporte, tiempo de ejecución y errores.

Servidores MCP existentes

El proyecto integra los servidores MCP de Filesystem y Git. Filesystem permite realizar operaciones sobre archivos únicamente dentro de un directorio previamente autorizado, limitando así el alcance de las acciones disponibles. Git MCP permite realizar operaciones de control de versiones, incluyendo la creación o utilización de un repositorio, staging de archivos y commits. Ambos servidores son ejecutados como procesos independientes y el cliente descubre sus herramientas dinámicamente mediante los métodos initialize y tools/list, evitando depender de una lista fija de herramientas

Servidor MCP propio local

El servidor MCP propio, denominado business-mcp-server, implementa el caso de uso de análisis ejecutivo. Su arquitectura separa la capa de protocolo MCP, las herramientas de negocio y el acceso a la base de datos. Entre las herramientas disponibles se encuentran funciones para obtener resúmenes de ventas, comparar períodos, analizar el rendimiento de productos, consultar métricas agrupadas y determinar la mezcla de productos por cliente. Cada herramienta define su nombre, descripción y esquema de entrada, valida los parámetros recibidos y ejecuta consultas previamente controladas sobre la fuente de datos.

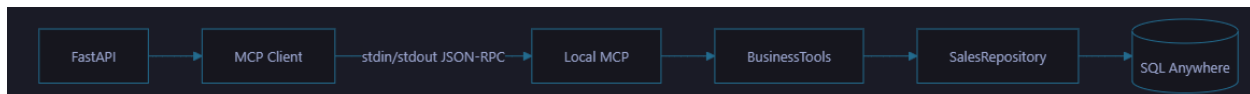


Figura 2. Diagrama servidor MCP propio local

Servidor MCP remoto

La versión remota reutiliza la misma lógica MCP del servidor local, pero cambia el mecanismo de transporte y la fuente de datos. El servicio se ejecuta mediante Docker Compose y utiliza PostgreSQL con un conjunto de datos anonimizado. El cliente se conecta mediante HTTP/HTTPS al endpoint remoto y utiliza la misma abstracción de cliente que en el modo local. Como medidas de seguridad, el servidor remoto puede exigir TLS, utiliza autenticación mediante Bearer token y configura PostgreSQL en modo de solo lectura, reduciendo el riesgo de modificaciones no autorizadas.

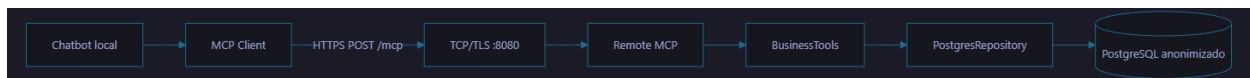


Figura 3. Diagrama servidor MCP remoto

Implementación de MCP y JSON-RPC

La comunicación MCP fue implementada manualmente utilizando JSON-RPC 2.0. Cada solicitud contiene la versión del protocolo, un identificador, un método y parámetros opcionales, mientras que cada respuesta conserva el mismo identificador y contiene un resultado o un objeto de error. El ciclo de vida comienza con initialize, seguido de notifications/initialized; posteriormente, el cliente utiliza tools/list para descubrir las herramientas y tools/call para ejecutarlas. Los identificadores de las solicitudes se generan mediante UUID y el cliente verifica que las respuestas correspondan con la solicitud que las originó..

Flujo End to end



Figura 4. Diagrama flujo end-to-end

Una consulta como “¿Cuáles fueron las ventas mensuales de 2025?” comienza en Vue y llega a FastAPI. La sesión agrega el mensaje al historial y el LLM recibe la pregunta junto con las tools. El modelo puede escoger `get_sales_summary`. El cliente genera el request JSON-RPC, el servidor valida las fechas, el repositorio ejecuta SQL parametrizado y el resultado retorna al LLM. Finalmente, el modelo redacta una explicación y FastAPI la entrega a Vue.

Testing

Se implementaron una serie de tests utilizando pytest. Los tests disponibles son:

- tests/test_jsonrpc.py.
- tests/test_mcp.py.
- tests/test_stdio.py.
- tests/test_llm.py.
- tests/test_sessions.py.
- tests/test_business_tools.py.
- tests/test_data_access.py.
- tests/test_postgres.py.
- tests/test_official_servers.py.
- tests/test_audit.py.
- tests/test_http_security.py.

Wireshark

tcp.port == 8080				
No.	Time	Source	Destination	Protocol Info
228	20:56:57.9569..	192.168.0.102	34.66.139.199	TCP 61434 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=253 [TCP PDU reassembled]
229	20:56:57.9572..	192.168.0.102	34.66.139.199	HTTP/3.. POST /mcp HTTP/1.1 , JSON (application/json)
231	20:56:58.0442..	34.66.139.199	192.168.0.102	TCP 8080 → 61434 [ACK] Seq=1 Ack=254 Win=64512 Len=0
232	20:56:58.0445..	34.66.139.199	192.168.0.102	TCP 8080 → 61434 [ACK] Seq=1 Ack=309 Win=64512 Len=0
234	20:56:58.0462..	34.66.139.199	192.168.0.102	HTTP/3.. HTTP/1.1 200 OK , JSON (application/json)
235	20:56:58.0498..	192.168.0.102	34.66.139.199	TCP 61434 → 8080 [FIN, ACK] Seq=308 Ack=157 Win=65280 Len=0
236	20:56:58.0838..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
238	20:56:58.1378..	34.66.139.199	192.168.0.102	TCP 8080 → 61434 [FIN, ACK] Seq=157 Ack=309 Win=64512 Len=0
239	20:56:58.1378..	192.168.0.102	34.66.139.199	TCP 61434 → 8080 [ACK] Seq=309 Ack=158 Win=65280 Len=0
240	20:56:58.1708..	34.66.139.199	192.168.0.102	8080 → 61435 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1412 SACK_PERM WS=9
241	20:56:58.1709..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
242	20:56:58.1717..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=253 [TCP PDU reassembled]
243	20:56:58.1718..	192.168.0.102	34.66.139.199	HTTP/3.. POST /mcp HTTP/1.1 , JSON (application/json)
244	20:56:58.2583..	34.66.139.199	192.168.0.102	TCP 8080 → 61435 [ACK] Seq=1 Ack=254 Win=64000 Len=0
245	20:56:58.2585..	34.66.139.199	192.168.0.102	TCP 8080 → 61435 [ACK] Seq=1 Ack=333 Win=64000 Len=0
246	20:56:58.2614..	34.66.139.199	192.168.0.102	TCP 8080 → 61435 [ACK] Seq=1 Ack=333 Win=64512 Len=1412 [TCP PDU reassembled in
247	20:56:58.2616..	34.66.139.199	192.168.0.102	HTTP/3.. HTTP/1.1 200 OK , JSON (application/json)
248	20:56:58.2617..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [ACK] Seq=333 Ack=2661 Win=65280 Len=0
249	20:56:58.2622..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [FIN, ACK] Seq=333 Ack=2661 Win=65280 Len=0
252	20:56:58.3497..	34.66.139.199	192.168.0.102	TCP 8080 → 61435 [FIN, ACK] Seq=2661 Ack=334 Win=64512 Len=0
253	20:56:58.3498..	192.168.0.102	34.66.139.199	TCP 61435 → 8080 [ACK] Seq=334 Ack=2662 Win=65280 Len=0
1154	20:57:16.0522..	192.168.0.102	34.66.139.199	TCP 56033 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1160	20:57:16.1382..	34.66.139.199	192.168.0.102	TCP 8080 → 56033 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1412 SACK_PERM WS=9
1161	20:57:16.1383..	192.168.0.102	34.66.139.199	TCP 56033 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1162	20:57:16.1388..	192.168.0.102	34.66.139.199	TCP 56033 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=254 [TCP PDU reassembled]
1163	20:57:16.1389..	192.168.0.102	34.66.139.199	HTTP/3.. POST /mcp HTTP/1.1 , JSON (application/json)
1166	20:57:16.2241..	34.66.139.199	192.168.0.102	TCP 8080 → 56033 [ACK] Seq=1 Ack=255 Win=64512 Len=0
1167	20:57:16.2244..	34.66.139.199	192.168.0.102	TCP 8080 → 56033 [ACK] Seq=1 Ack=518 Win=64512 Len=0
1168	20:57:16.2272..	34.66.139.199	192.168.0.102	HTTP/3.. HTTP/1.1 200 OK , JSON (application/json)
1169	20:57:16.2291..	192.168.0.102	34.66.139.199	TCP 56033 → 8080 [FIN, ACK] Seq=518 Ack=265 Win=65024 Len=0
1175	20:57:16.3153..	34.66.139.199	192.168.0.102	TCP 8080 → 56033 [FIN, ACK] Seq=265 Ack=519 Win=64512 Len=0
1176	20:57:16.3154..	192.168.0.102	34.66.139.199	TCP 56033 → 8080 [ACK] Seq=519 Ack=266 Win=65024 Len=0
1187	20:57:17.4641..	192.168.0.102	34.66.139.199	TCP 56036 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1189	20:57:17.5532..	34.66.139.199	192.168.0.102	TCP 8080 → 56036 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1412 SACK_PERM WS=9
1190	20:57:17.5533..	192.168.0.102	34.66.139.199	TCP 56036 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1191	20:57:17.5536..	192.168.0.102	34.66.139.199	TCP 56036 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=254 [TCP PDU reassembled]
1192	20:57:17.5536..	192.168.0.102	34.66.139.199	HTTP/3.. POST /mcp HTTP/1.1 , JSON (application/json)
1193	20:57:17.6423..	34.66.139.199	192.168.0.102	TCP 8080 → 56036 [ACK] Seq=1 Ack=255 Win=64512 Len=0
1194	20:57:17.6427..	34.66.139.199	192.168.0.102	TCP 8080 → 56036 [ACK] Seq=1 Ack=494 Win=64512 Len=0
1199	20:57:18.1099..	34.66.139.199	192.168.0.102	HTTP/3.. HTTP/1.1 200 OK , JSON (application/json)
1200	20:57:18.1103..	192.168.0.102	34.66.139.199	TCP 56036 → 8080 [FIN, ACK] Seq=494 Ack=484 Win=65024 Len=0
1205	20:57:18.1999..	34.66.139.199	192.168.0.102	TCP 8080 → 56036 [FIN, ACK] Seq=484 Ack=495 Win=64512 Len=0
1206	20:57:18.2000..	192.168.0.102	34.66.139.199	TCP 56036 → 8080 [ACK] Seq=495 Ack=485 Win=65024 Len=0
2348	20:57:40.6333..	192.168.0.102	34.66.139.199	TCP 63600 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2351	20:57:40.7215..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1412 SACK_PERM WS=9
2352	20:57:40.7215..	192.168.0.102	34.66.139.199	TCP 63600 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
2353	20:57:40.7218..	192.168.0.102	34.66.139.199	TCP 63600 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=254 [TCP PDU reassembled]
2354	20:57:40.7219..	192.168.0.102	34.66.139.199	HTTP/3.. POST /mcp HTTP/1.1 , JSON (application/json)
2355	20:57:40.8093..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [ACK] Seq=1 Ack=255 Win=64512 Len=0
2356	20:57:40.8096..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [ACK] Seq=1 Ack=453 Win=64512 Len=0
2357	20:57:40.9177..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [ACK] Seq=1 Ack=453 Win=64512 Len=1412 [TCP PDU reassembled in
2358	20:57:40.9181..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [PSH, ACK] Seq=1413 Ack=453 Win=64512 Len=2824 [TCP PDU reasse
2359	20:57:40.9181..	34.66.139.199	192.168.0.102	HTTP/3.. HTTP/1.1 200 OK , JSON (application/json)
2360	20:57:40.9181..	192.168.0.102	34.66.139.199	TCP 63600 → 8080 [ACK] Seq=453 Ack=4993 Win=65280 Len=0
2361	20:57:40.9185..	192.168.0.102	34.66.139.199	TCP 63600 → 8080 [FIN, ACK] Seq=453 Ack=4993 Win=65280 Len=0
2367	20:57:41.0059..	34.66.139.199	192.168.0.102	TCP 8080 → 63600 [FIN, ACK] Seq=4993 Ack=454 Win=64512 Len=0

Initialize

```
.....
-4-.....^.....E-
-..M@-.....f"B
-.....r-.....~>#P-
-..o-..{" jsonrpc"
: "2.0", " id": "167
51243535 04a3a9b8
ffe1d0e5 b02b6", "
method": "initial
ize", "pa rams": {"
protocol Version"
: "2024-1 1-05", "c
apabilit ies": {},
"clientI nfo": {"n
ame": "ph ase2-cli
ent", "ve rsion": "
0.1.0"}} }
```

notifications/initialized

```
.....
-4-.....^.....E-
-..^S@-.....f"B
-.....z-..P-
-..oh-..{" jsonrpc"
: "2.0", " method":
"notific ations/i
nitializ ed"}
```

El cliente envía una solicitud JSON-RPC initialize mediante HTTP POST al endpoint remoto. Este mensaje corresponde al inicio del proceso de sincronización entre el cliente y el servidor MCP.

Tools/list

```
.....
-4-.....^.....E-
-..wY@-.....f"B
-....."- d:B-..P-
-..o-..{" jsonrpc"
: "2.0", " id": "f19
184b7445 34d4aa10
24559476 bbce7", "
method": "tools/l
ist"}
```

El cliente solicita el catálogo de herramientas.

```

~....."s tring"},
"product _code":{
"type":" string"}
,"limit" :{"type"
:"intege r","mini
mum":1," maximum"
:500},"o rder_by"
:{"type" : "string
"}}, {" name": "g
et_sales _summary
","descr iption":
"Summari ze real
sales by month.
Dates mu st be st
rings in YYYY-MM
-DD form at.", "in
putSchem a":{"typ
e":"obje ct","req
uired":[ "start_d
ate","en d_date"]
},"proper ties":{"
start_da te":{"ty
pe":"str ing","fo
rmat":"d ate"},"e
nd_date" :{"type"
:"string ", "forma
t":"date "},"curr
ency":{" type":"s
tring"," enum":["
Q.", "US$ "]}}, {
"name": "compare_
periods", "descri
ption": " Compare
sales me trics fo
r two pe riods. E
very dat e must b
e a YYYY -MM-DD s
tring.", "inputSc
hema":{" type":"o
bject"," required
":["firs t_period
","secon d_period
"],"prop erties":
{"first_ period":
{"type": "object"
},"secon d_period
":{"type ":"objec
t"}}, {" name": "

```

El servidor le responde con las herramientas disponibles.

Tools/call

Cuando el LLM necesita consultar datos empresariales se envía

```
..p9..{" jsonrpc":  
:"2.0", "id": "f74  
ef6fabfa b4442962  
6733a1ac 55e22", "  
method": "tools/c  
all", "pa rams": {"  
name": "q uery_bus  
iness_me trics", "  
argument s": {"sta  
rt_date": "2025-0  
1-01", "e nd_date"  
:"2025-1 2-31", "g  
roup_by": ["clien  
t"], "met rics": ["  
units"], "limit":  
10, "ord[er_by": "u  
nits DES C"}}}
```

En este caso se indica que se necesitan fechas para poder ejecutar la consulta.

```
.....  
^.....4 .....E..  
0..@7... "B....  
f.....4 ..mCJ.P..  
~.....HT TP/1.1 2  
00 OK..d ate: Mon  
, 07 Sep 2026 02  
:57:14 G MT..serv  
er: uvic orn..con  
tent-len gth: 138  
..conten t-type:  
applicat ion/json  
.....{"js onrpc": "  
2.0", "id ": "f74ef  
6fabfab4 44296267  
33a1ac55 e22", "er  
ror": {"c ode": -32  
602, "mes sage": "o  
rder_by must be  
one of t he selec  
ted metr ics"}}}
```

```
.....  
4.....^ .....E..  
...f@.....f"B  
.....R !Y...8P..  
..p!..{" jsonrpc":  
:"2.0", "id": "767  
28cc4c0a b418a848  
b11d152c ba258", "  
method": "tools/c  
all", "pa rams": {"  
name": "q uery_bus  
iness_me trics", "  
argument s": {"sta  
rt_date": "2025-0  
1-01", "e nd_date"  
:"2025-1 2-31", "g  
roup_by": ["clien  
t"], "met rics": ["  
units"], "limit":  
10}}}
```

```
applicat ion/json  
.....{"js onrpc": "  
2.0", "id ": "76728  
cc4c0ab4 18a848b1  
d152cba 258", "re  
sult": {" content"  
: [{"type ": "text"  
, "text": "{ \"star  
t_date\" : \"2025  
-01-01\", \"end_  
date\" : \"2025-1  
2-31\", \"rows\"  
: [{ \"cli ent\" :  
null, \" units\" :  
\"14412 28.78543  
173\" } ] } } } }  
\"}], \"str  
ucturedC ontent\" :  
{ \"start_ date\" : \"2  
025-01-0 1\", \"end_  
date\" : \"2 025-12-3  
1\", \"rows \" : [{ \"cli  
ent\" : nul l, \"units  
\" : \"14412 28.78543  
173\" } ] } } } }
```

El servidor responde con los datos después de especificar las fechas

Análisis por Capas

Identificación de los hosts y puertos

Elemento	Valor
Cliente MCP	192.168.0.102
Servidor MCP remoto	34.66.139.199
Puerto del servidor MCP	8080/TCP
Puerto del cliente	Puerto efímero, por ejemplo 61433, 61435 o 56036
Protocolo de transporte	TCP
Protocolo de aplicación	HTTP/1.1
Formato de mensajes MCP	JSON-RPC 2.0
Endpoint observado	POST /mcp

Establecimiento de la conexión TCP

Antes de enviar los mensajes MCP, cliente y servidor deben establecer una conexión confiable utilizando TCP. En la captura se observa el *three-way handshake* compuesto por los frames 214, 215 y 216.

Frame 214 - SYN del cliente

El frame 214 corresponde al primer paso del establecimiento de la conexión TCP. El cliente 192.168.0.102, utilizando el puerto efímero 61433, envía un segmento al servidor 34.66.139.199 en el puerto 8080 con la bandera SYN activa.

Capa de enlace

En la capa de enlace, el paquete IP es encapsulado en una trama correspondiente a la tecnología de red utilizada por el equipo, ya sea Ethernet o Wi-Fi. Esta capa se encarga de transportar la trama dentro del segmento local mediante direcciones MAC. Debido a que el servidor se encuentra fuera de la red local, la dirección MAC de destino no corresponde directamente al servidor remoto, sino al siguiente salto de la red, normalmente el gateway o router local.

Capa de red

En la capa de red se utiliza IP para permitir el direccionamiento entre el cliente y el servidor remoto. En este frame se observa:

- IP origen: 192.168.0.102
- IP destino: 34.66.139.199

La dirección 192.168.0.102 pertenece a una red privada, por lo que el tráfico debe atravesar el gateway de la red local antes de alcanzar el servidor remoto.

Capa de transporte

En la capa de transporte se utiliza TCP:

- Puerto origen: 61433
- Puerto destino: 8080
- Bandera: SYN

La bandera SYN indica que el cliente desea iniciar una nueva conexión y comenzar la sincronización de números de secuencia.

Capa de aplicación

En este frame todavía no existe información MCP ni JSON-RPC. El paquete pertenece exclusivamente al establecimiento de la conexión TCP.

Frame 215 - SYN/ACK del servidor

El frame 215 representa el segundo paso del *three-way handshake*. El servidor remoto responde al cliente utilizando las banderas SYN y ACK.

Capa de enlace

La trama viaja desde la infraestructura de red hacia el equipo cliente. En el último segmento de la comunicación local, la capa de enlace entrega la trama al cliente mediante su dirección MAC.

Capa de red

Las direcciones IP se encuentran invertidas con respecto al frame anterior:

- IP origen: 34.66.139.199
- IP destino: 192.168.0.102

Esto confirma que la respuesta proviene del servidor remoto hacia el cliente.

Capa de transporte

Los puertos también se encuentran invertidos:

- Puerto origen: 8080
- Puerto destino: 61433

- Banderas: SYN, ACK

El servidor utiliza ACK para confirmar la recepción del SYN enviado por el cliente y utiliza su propio SYN para sincronizar su número de secuencia con el cliente.

Capa de aplicación

Todavía no existe tráfico de aplicación. El propósito del segmento continúa siendo establecer la conexión TCP.

Frame 216 - ACK del cliente

El frame 216 representa el tercer y último paso del *three-way handshake*. El cliente responde al servidor con un segmento que contiene la bandera ACK.

Capa de enlace

La trama vuelve a ser enviada desde el cliente hacia el gateway local para continuar su recorrido hasta el servidor remoto.

Capa de red

Las direcciones observadas son:

- IP origen: 192.168.0.102
- IP destino: 34.66.139.199

Capa de transporte

Se observa:

- Puerto origen: 61433
- Puerto destino: 8080
- Bandera: ACK

Este ACK confirma la recepción del SYN, ACK del servidor. Después de este frame, la conexión TCP queda establecida y puede comenzar el intercambio de datos de aplicación.

El establecimiento puede representarse de la siguiente forma:

Cliente 192.168.0.102 Servidor 34.66.139.199:8080

SYN

----->

SYN + ACK

<-----

ACK

----->

Conexión TCP establecida

Solicitud MCP

Frame 243 — HTTP POST /mcp

El frame 243 corresponde a una solicitud enviada desde el cliente hacia el servidor MCP remoto. Wireshark identifica el paquete como:

POST /mcp HTTP/1.1, JSON

La comunicación tiene la siguiente dirección:

- Origen: 192.168.0.102
- Destino: 34.66.139.199
- Puerto destino: 8080

En los datos del paquete se observa un request HTTP similar a:

POST /mcp HTTP/1.1

Host: 34.66.139.199:8080

Accept-Encoding: gzip, deflate, zstd

Connection: keep-alive

User-Agent: python-httpx/0.28.1

Content-Type: application/json

Accept: application/json

Authorization: Bearer ...

Content-Length: ...

Después de los encabezados se encuentra el mensaje JSON-RPC enviado por el cliente. En la captura se identifica un objeto con la estructura:

```
{  
  "jsonrpc": "2.0",  
  "id": "f19184b744534d4aa1024559476b6bce7",  
  "method": "tools/list"  
}
```

El token Bearer fue omitido deliberadamente del análisis por razones de seguridad.

El método tools/list corresponde a una **solicitud MCP de descubrimiento de herramientas**. El cliente solicita al servidor la lista de tools que puede utilizar.

Esta operación ocurre después de que la sesión MCP ha sido inicializada y permite al host conocer:

- nombre de cada tool;
- descripción;
- parámetros aceptados;
- esquema de entrada (inputSchema).

Por lo tanto, el frame 243 se clasifica como:

Solicitud / petición MCP

y no como mensaje de sincronización.

Capa de enlace

El request es encapsulado dentro de una trama de enlace para ser enviado desde el cliente hacia el gateway de la red local. La capa de enlace se encarga únicamente de la transmisión dentro del enlace físico o inalámbrico actual.

Capa de red

El paquete IP contiene:

- IP origen: 192.168.0.102
- IP destino: 34.66.139.199

IP permite encaminar la información desde el cliente hasta el servidor remoto.

Capa de transporte

HTTP se transporta utilizando TCP. La conexión utiliza un puerto efímero del cliente y el puerto 8080 del servidor. TCP proporciona entrega confiable y ordenada de los bytes que contienen el mensaje HTTP y JSON-RPC.

Capa de aplicación

En esta capa se observan tres elementos:

1. **HTTP/1.1**, utilizado para transportar la solicitud.
2. **JSON-RPC 2.0**, utilizado como formato estructurado de request y response.
3. **MCP**, que define el significado del método tools/list.

La encapsulación lógica puede representarse como:

MCP: tools/list, JSON-RPC 2.0, HTTP POST /mcp,TCP :8080,IP,Ethernet / Wi-Fi

Respuesta MCP

Frame 247 — HTTP/1.1 200 OK

El frame 247 corresponde a la respuesta del servidor al request tools/list. Wireshark identifica el paquete como:

HTTP/1.1 200 OK, JSON

La dirección de comunicación es ahora:

- Origen: 34.66.139.199
- Destino: 192.168.0.102
- Puerto origen: 8080
- Puerto destino: puerto efímero del cliente

En los datos del paquete se observa el encabezado:

HTTP/1.1 200 OK

Content-Type: application/json

...

seguido de una respuesta JSON-RPC cuya estructura comienza de forma equivalente a:

```

{
  "jsonrpc": "2.0",
  "id": "f19184b744534d4aa1024559476b6bce7",
  "result": {
    "tools": [
      ...
    ]
  }
}

```

Dentro del campo result se observa la lista de herramientas disponibles, incluyendo información como:

- nombre de la tool;
- descripción;
- esquema de entrada;
- parámetros requeridos;
- tipos de datos admitidos.

En la captura se puede observar, por ejemplo, la definición de la tool `get_customer_product_mix`.

Correlación request-response

Uno de los elementos más importantes de JSON-RPC es que la respuesta utiliza el mismo id del request original.

En este intercambio se observa:

Request:

id = f19184b744534d4aa1024559476b6bce7

method = tools/list

y posteriormente:

Response:

id = f19184b744534d4aa1024559476b6bce7

```
result = { tools: [...] }
```

La coincidencia del identificador permite al cliente correlacionar cada respuesta con la solicitud que la originó, incluso cuando existen múltiples solicitudes en ejecución.

Clasificación

El frame 247 se clasifica como:

Respuesta MCP

El código HTTP 200 OK indica que la solicitud HTTP fue procesada correctamente, mientras que el campo JSON-RPC result indica que la ejecución lógica de la solicitud también produjo un resultado válido.

Capa de enlace

La trama es recibida en la interfaz de red del cliente y entregada a la capa superior correspondiente.

Capa de red

El paquete viaja en sentido inverso:

- IP origen: 34.66.139.199
- IP destino: 192.168.0.102

Capa de transporte

TCP garantiza la entrega ordenada del contenido HTTP desde el servidor hacia el cliente. El servidor utiliza el puerto 8080 y responde al puerto efímero que había sido abierto por el cliente.

Capa de aplicación

En la capa de aplicación se encuentra:

```
HTTP/1.1 200 OK, JSON-RPC 2.0 Response, result.tools[]
```

El resultado contiene las capacidades disponibles en el servidor MCP que posteriormente pueden ser utilizadas mediante tools/call.

Dificultades y soluciones

La primera dificultad que surgió en el proyecto fue con la base de datos, al ser una base de datos bastante grande en temas de tablas se tuvo que consultar al encargado de la base de datos para poder tener un entendimiento de las relaciones que existen.

La segunda dificultad que surgió fue la anonimización de la BD para el mcp remoto. Debido a que la base de datos original pesaba alrededor de 24GBs y era de SQL Anywhere, era bastante complicado trasladarla a la instancia remota. Adicional a esto, se necesitaba anonimizar los datos para protegerlos, ya que pertenecen a una empresa real y es información valiosa. Para esto decidí realizar una anonimización de una porción de la base de datos, correspondiente al año 2025. La base de datos se trasladó a un json que fue más fácil de trasladar y cargar a la instancia de Google Cloud.

La última dificultad que surgió fue la comunicación entre la instancia remota y el programa como tal. Al principio pensé que era un error de configuración o algún error en mi código, pero resultó ser un tema de configuraciones de firewall en Google. Después de crear la regla del firewall en la instancia, la conexión fue exitosa.

Conclusiones

1. MCP permite separar el razonamiento del LLM de la ejecución de operaciones concretas y controladas.
2. JSON-RPC proporciona un contrato simple para sincronizar cliente y servidor, correlacionar requests mediante IDs y representar errores.
3. La misma abstracción de cliente puede utilizar stdio en modo local y HTTP/HTTPS en modo remoto; lo que cambia principalmente es el transporte y el repositorio de datos. La validación de parámetros y el SQL parametrizado son esenciales para evitar que una interfaz de lenguaje natural se convierta en acceso arbitrario a la base de datos.
4. El logging bidireccional permite demostrar el flujo MCP y facilita el diagnóstico de errores.
5. El transporte remoto introduce requisitos adicionales de autenticación, TLS, puertos, despliegue y control de permisos.

Referencias

Amazon Web Services. (s. f.). What's the Difference Between HTTP and HTTPS? Amazon Web Services, Inc. <https://aws.amazon.com/compare/the-difference-between-https-and-http/>

Google Cloud Documentation. (s. f.). Google Cloud Documentation. <https://docs.cloud.google.com/docs>

Inc, D. (2026, 12 agosto). Home. Docker Documentation. <https://docs.docker.com/>

Model context protocol. (s. f.). GitHub. <https://github.com/modelcontextprotocol>

OpenAI API Platform Documentation. (s. f.). OpenAI Developers. <https://developers.openai.com/api/docs>

PostgreSQL: documentation. (s. f.). The PostgreSQL Global Development Group. <https://www.postgresql.org/docs/>

pytest documentation. (s. f.). <https://docs.pytest.org/en/stable/>

SAP Help Portal | SAP Online Help. (s. f.). https://help.sap.com/docs/SAP_SQL_Anywhere

Team, A. P. (s. f.). What is JSON-RPC 2.0: A Comprehensive Guide - A2A Protocol Documentation | A2A Protocol. A2A Protocol. <https://www.a2aprotocol.org/en/docs/json-rpc-2-0>

Wireshark • Go Deep | Documentation. (s. f.). Wireshark. <https://www.wireshark.org/docs/>